Health Checks

who gets traffic, and how not to remove everyone at once

Watch the 4-min video

In one line: health checks decide which backends get traffic: active checks probe an endpoint, passive checks watch real errors; keep liveness shallow, use readiness to hold traffic until an instance can really serve, and beware deep checks on shared dependencies, which can remove every server at once.

1 Two ways to fail

A server is up and answers pings, but every real request returns an error, and the load balancer keeps sending it a third of your traffic. Or the opposite: one database hiccup, and the balancer marks *every* server unhealthy at once. Health checks decide who gets traffic, so both mistakes hurt.

2 Active and passive checks

Active checks: the load balancer calls an endpoint such as `GET /health` every few seconds. After a few failures in a row (the *unhealthy threshold*) it removes the server; after a few successes in a row (the *healthy threshold*) it adds it back.

Passive checks, also called **outlier detection**, watch real traffic: if a backend returns too many errors or timeouts, it is **ejected** for a while, then tried again. Passive checks catch problems a health endpoint can't see, such as one bad code path. Use both.

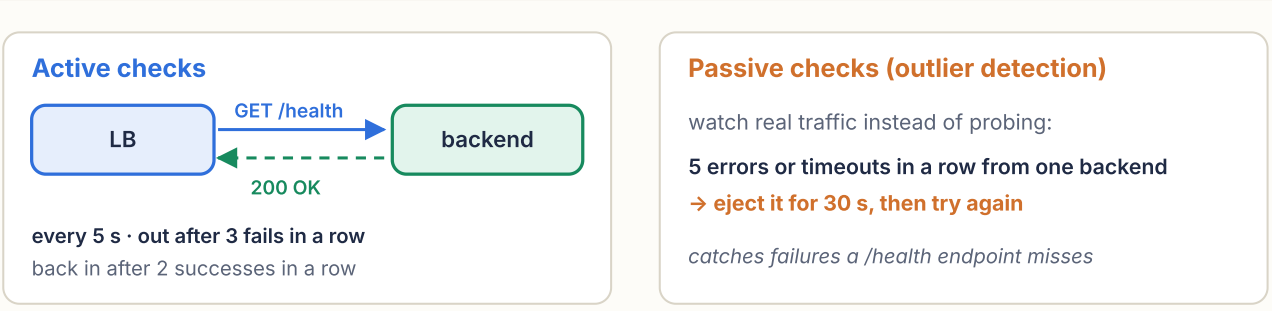


Figure 1. Active checks probe an endpoint on a schedule; passive checks judge each backend by the real responses it sends.

3 Liveness, readiness and startup probes

Kubernetes makes a distinction that is useful everywhere:

- **Liveness**: is this process alive, or stuck (deadlocked, out of memory)? If it fails, the container is **restarted**.
- **Readiness**: can this instance serve traffic *right now*? If it fails, the pod is **removed from the Service's endpoints**, but not restarted.
- **Startup**: gives a slow-starting app time; liveness and readiness only begin once it passes.

WORKED EXAMPLE: A 40-SECOND CACHE WARM-UP

A service starts and spends 40 seconds loading a large cache into memory. Its liveness probe passes, because the process is fine. Its readiness probe fails until the cache is warm, so it receives no traffic yet. Once loaded, readiness passes and traffic flows in. If you had used liveness for this check, Kubernetes would kill and restart the container every time, forever: a crash loop.

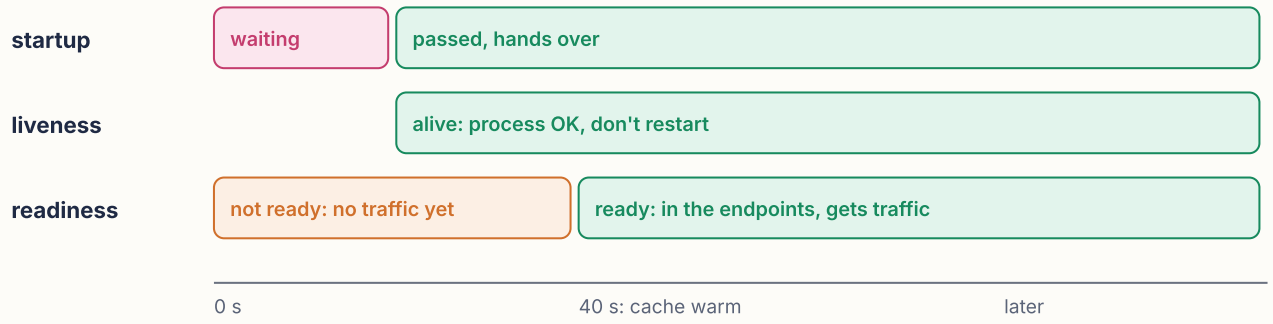
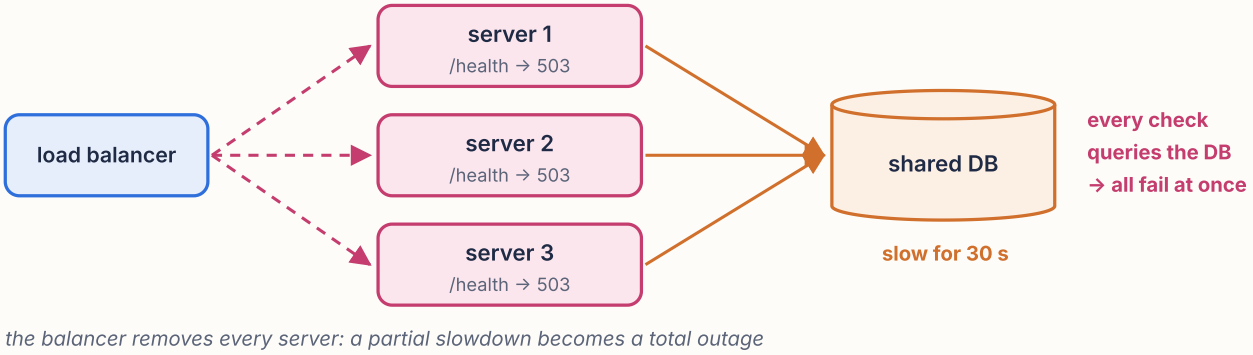


Figure 2. The three probes over a pod's start-up: alive all along, but ready only once the cache is warm.

4 How deep should a check go?

A **shallow** check confirms the process responds. A **deep** check also queries the database, the cache and downstream services. Deep checks catch more problems, but they are dangerous: if the **shared** database slows down, every instance fails its check at the same moment, the load balancer removes all of them, and a partial slowdown becomes a total outage. Worse, the app could still have served cached pages or a friendly error.



the balancer removes every server: a partial slowdown becomes a total outage

Figure 3. Deep checks on a shared dependency: one slow database makes every server look unhealthy at once.

5 Guidelines

- **Liveness: shallow.** Only the process itself. Never the database.
- **Readiness: what the instance owns.** Its warm cache, its config, its local disk. Be careful with shared dependencies.
- **Handle a failing dependency inside the app** with errors, timeouts and degraded responses, instead of disappearing from the pool.
- **Fail open.** If too many hosts look unhealthy, send traffic to all of them anyway. Envoy's *panic threshold* (default 50%) does this.
- **Tune the numbers.** Intervals of a few seconds and thresholds of 2 or 3 avoid *flapping* (in, out, in, out).
- **Make health endpoints cheap**, and exclude them from rate limits and auth.
- **Graceful shutdown:** on SIGTERM, fail readiness first, finish in-flight requests, then exit.

COMMON MISTAKES

- A liveness probe that checks the database, so a DB blip restarts every pod.
- `/health` that always returns 200, even when the app can't serve.
- No startup or readiness probe on a slow starter, so it gets traffic before it's ready, or is killed while booting.
- Thresholds of 1, so one slow response removes a server.

6 Key terms

Active check: the balancer probes an endpoint on a schedule

Passive check: judging backends by real responses

Outlier ejection: temporarily removing a misbehaving backend

Liveness probe: is the process alive? fail → restart

Readiness probe: can it serve now? fail → no traffic

Startup probe: extra time for slow starters

Flapping: a backend bouncing in and out of the pool

Panic threshold: too many unhealthy → send to all

7 Interview questions

Q1. Should a health check verify the database? BASIC

Usually not for liveness, and only carefully for readiness, because the database is shared by every instance; a DB problem should be handled in the app, not by removing every server.

Q2. Why can deep health checks be dangerous? BASIC

If a shared dependency slows down, every instance fails at once and the load balancer removes them all, turning a partial problem into a full outage.

Q3. What's the difference between liveness and readiness? BASIC

Liveness asks if the process is alive; failing it restarts the container. Readiness asks if it can serve traffic now; failing it removes the pod from the endpoints without restarting it.

Q4. A new pod takes 60 s to start and keeps restarting. Why? SCENARIO

The liveness probe fails before the app is up, so Kubernetes kills it. Add a startup probe (or a longer initial delay) and use readiness to hold traffic until it's ready.

Q5. Health checks pass, but users see errors from one server. How do you catch that? SCENARIO

Add passive health checking (outlier detection) so a backend with a high error or timeout rate on real traffic is ejected, and make the active check exercise the real serving path.

Q6. How do you shut down a server without dropping requests? TRICKY

On SIGTERM, start failing readiness so the balancer stops sending new requests, keep serving in-flight ones (connection draining, a grace period), then exit.

Next: SD-G-008 · Service Discovery backendroadmap.com